

为什么要把“结果”写在“参数”里？

当你定义 `void getMaxAndMin(int arr[], int len, int* max, int* min)` 时，你的大脑可能在想：

- `arr` 是我给函数的（输入）
- `len` 是我给函数的（输入）
- 结果应该从函数“跳”出来，而不是我塞进去。

1. 核心矛盾：C 语言的“单出口”限制

在 C 语言中，`return` 就像是一个只能容纳一个人的小单间。

- 如果你只需要最大值，你可以写 `int getMax(...)`，让最大值通过 `return` 走出来。
- 但现在你有两个结果（最大和最小）。`return` 的门太窄了，一次只能走出一个。

2. 破局思路：提供“储物柜”的地址

既然结果不能从 `return` 门里走出来，我们就换个思路：在函数外面准备好两个空的储物柜，把柜子的地址告诉函数。

我们可以这样拆解你的思维过程：

- **第一步（输入）：**我要处理哪个数组？长度是多少？
 - 于是你写了：`int arr[], int len`
- **第二步（诉求）：**我希望函数把“最大值”放在哪？把“最小值”放在哪？
 - 你在 `main` 函数里定义了 `int p1, p2;`。这里的 `p1` 和 `p2` 就是那两个储物柜。它们定义在 `getMaxAndMin` 函数之外，所以函数执行完后，它们依然存在。
- **第三步（授权）：**为了让函数能把结果放进 `p1` 和 `p2`，你必须在调用函数时，把这两个柜子的地址传给 `getMaxAndMin`。
 - 在 `main` 中写下 `getMaxAndMin(arr, len, &p1, &p2);`。
 - 这里的 `&p1` 就是告诉函数：“去这个地址找我的柜子”。
 - 此时，`getMaxAndMin` 函数内部的指针 `max` 就拿到了 `p1` 的地址，`min` 就拿到了 `p2` 的地址。
- **第四步（执行）：通过地址实时修改**
 - 函数内部并不一定等到最后才给结果。
 - `*max = arr[0];` —— 这一行代码执行时，函数顺着地址找到了外面的 `p1`，直接把第一个数放了进去。
 - `*max = arr[i];` —— 循环中每次找到更大的数，函数都会顺着地址把 `p1` 里的旧值擦掉，写入新值。
 - **这就是为什么不用 `return`：**因为函数是在直接操作你提供的外部变量。

3. 生活中的类比：快递员上门

想象你请了一个**维修工（函数）**来家里修东西：

- 1. **输入参数**：你告诉他坏了的机器在哪里（数组）以及有多少零件（长度）。
- 2. **传统的 return**：维修工修好后，在门口递给你一个修好的零件。
- 3. **指针作为输出参数**：你直接带维修工去你的**工具箱（变量 p1 和 p2）**前，指着它们说：“修好后的零件，直接放进这两个箱子里。”
 - 这里的“指着箱子”就是传递地址 `&p1` 。此时维修工在自己的笔记上记下了这两个箱子的位置，这个笔记就是参数里的 `int* max` 。

4. 这种思路的好处

- **突破限制**：你想返回 2 个、5 个甚至 10 个值都可以，只要你提供足够多的“箱子地址”。
- **效率高**：如果返回的结果是一个巨大的数据块，通过指针直接写入比通过 `return` 拷贝整个物体要快得多。

5. 什么时候不需要写？

如果你只需要一个结果，或者你使用 `struct` 将多个结果包装成了一个整体，那么你可以只写到 `int len` 就结束，然后通过 `return` 返回。

6. 终极疑惑：什么时候加星号（*）？

这是新手最容易混淆的地方，请看下表对照：

代码	含义	角色	是否需要 *
<code>int p1 = 10;</code>	定义一个普通的整数变量	柜子本身 (Owner)	不需要
<code>int *max;</code>	定义一个指针变量	存地址的纸条 (Note)	需要 (声明我是指针)
<code>*max = 20;</code>	“去这个地址对应的柜子里放个20”	远程操作 (Action)	需要 (解引用/动手)
<code>&p1</code>	“获取 p1 的地址”	柜子的位置信息	不需要 *, 需要 &

记住：只有当你是在定义“地址变量”，或者通过“地址”去操作别人的时候，才需要星星。如果不涉及地址，只是普通存数，就不需要星星。

CAN 的总结： 在 C 语言中，看到参数列表里的指针（尤其是没有 `const` 修饰的），你可以立刻反应过来：“哦！这是函数要往里写结果的地方，是它的‘输出口’。”